

Back To TOP

No. My SQL Questions

[What is MySQL](#)

[What is MySQL?](#)

[What is Sql?](#)

[What is the difference between SQL and MySQL?](#)

[Difference between SQL and NoSQL?](#)

No. Mysql User Management

[Create Databse user?](#)

[Show Databse user?](#)

[Show Current user?](#)

[User Password Change?](#)

[Drop User?](#)

[Grant Privileges to the MySQL New User?](#)

[Show Privileges?](#)

[REVOKE Privileges?](#)

No. Database

[What is a database?](#)

[Database:- Show](#)

[Database:- Create](#)

[Database:- Rename](#)

[Database:- Drop/Delete](#)

[Database:- Select](#)

No. Other Questions

[What is ACID property/SQL TRANSACTIONS](#)

[Difference between CHAR vs VARCHAR](#)

[Difference between Delete, Truncate & Drop?](#)

[Difference between WHERE and HAVING clauses?](#)

[SQL Comments?](#)

[What is Aggregate function?](#)

No. Other Questions

constraints

Wildcard Characters/Like Query

Aliases?

No. Keys

Primary Key?
primary Key:- Add
primary Key:- Delete
Unique Key?
ALTER unique key?
Drop unique key?
Difference between Primary Key & Unique Key?
Foreign Key?
Foreign Key Add/ALTER?
DROP Foreign Key?
Composite Key?
Difference between Primary Key & Foreign Key?

No. Joins

What Is Joins?
self join
INNER JOIN
Left JOIN/LEFT OUTER JOIN
Right JOIN
Outer join
CROSS Join
Full Join/FULL OUTER JOIN

Index**No. Index**

What is Index?
Types of Indexes
Unique Indexes
Show Index
Alter/Modify an Index
Drop Index

No. Index

Unique Indexes

Cluster Index

Non cluster index

difference between cluster and non cluster index?

view**No. View**

What is View?

Create view

Show view

Alter view

Deleted view

Views used in real projects?

No.

What Is Union & Union All?

Difference between Union & Union All?

What is MINUS?

What is EXCEPT?

What is Intersect?

What is MySQL?

- MySQL is an open-source relational **database management system** (RDBMS) that uses SQL to store, manage, and retrieve data.
- It's commonly used for managing data in web applications and is known for its performance and ease of use.

 [Back to Top](#)

What is Sql?

- SQL stands for **structure query language**.
- SQL (Structured Query Language) is a standard language used to **create, read, update, and delete** data in relational databases. It is also used to create and modify database structures such as tables.
- SQL language hai, database nahi.

What is the difference between SQL and MySQL?

- SQL is a standard language used to communicate with relational databases, whereas MySQL is an RDBMS that uses SQL to store, manage, and retrieve data.
- SQL → Language
- MySQL → RDBMS / Database Management System

MySQL vs NoSQL?

- **MySQL** is a **relational database** that stores data in tables with **predefined schemas** and uses SQL for querying.
- **NoSQL** is a **non-relational database** that stores data in **flexible formats such as documents, key-value pairs, graphs, or columns**, making it suitable for large-scale and rapidly changing applications.

When to Use MySQL?

- Banking Systems
- E-commerce Orders
- ERP/CRM Applications
- Applications requiring complex JOINS and transactions

When to Use NoSQL?

- Social Media Apps
- Real-time Analytics
- Chat Applications
- Large-scale Distributed Systems

MySQL	NoSQL
Relational Database	Non-Relational Database
Data tables mein store hota hai	Data documents, key-value, graph ya columns mein store hota hai
Fixed Schema	Flexible Schema
SQL language use karta hai	Different query methods use karta hai
JOIN support karta hai	JOIN support limited ya nahi hota
ACID transactions strong hote hain	Mostly scalability par focus
Vertical Scaling (RAM/CPU badhao)	Horizontal Scaling (servers badhao)
Complex relationships ke liye best	Large-scale data ke liye best
Data consistency high	High availability aur performance
Example: MySQL, PostgreSQL	MongoDB, Redis, Cassandra

Difference between SQL and NoSQL?

- The main difference between SQL (Structured Query Language) and NoSQL (Not Only SQL) databases lies in how they store, manage, and retrieve data.
- Here's a breakdown of the key differences:

1. Data Structure:

- SQL:
 - Relational databases use tables to store data in rows and columns.
 - Data is organized into predefined schemas with relationships (foreign keys) between tables.
 - Examples: MySQL, PostgreSQL, Oracle, MS SQL Server.
- NoSQL:
 - Non-relational databases can store data in various formats such as key-value pairs, document-based, column-family, or graph-based.
 - More flexible schema (often schema-less or dynamic schemas), allowing changes in data structure without affecting the database.
 - Examples: MongoDB (document), Redis (key-value), Cassandra (column-family), Neo4j (graph).

2. Schema:

- SQL:
 - Strict schema: data must adhere to a predefined structure (tables, columns, data types).
 - Changes to schema (adding/removing columns) can be complex.
- NoSQL:
 - Flexible schema: no strict schema, data can be stored with varying structures.
 - Allows quick changes to the data model, making it ideal for rapidly evolving applications.

3. Scalability:

- SQL:
 - Vertical scaling (scaling up by upgrading hardware resources like CPU, RAM).
 - More challenging to scale horizontally (across multiple machines).
- NoSQL:
 - Horizontal scaling (scaling out by adding more servers or nodes).
 - Designed to handle large volumes of data and high-traffic loads more efficiently.

4. Transactions & Consistency:

- SQL:
 - ACID (Atomicity, Consistency, Isolation, Durability) properties are strictly followed to ensure data integrity and reliability.
 - Best suited for applications where data consistency is critical (e.g., banking, financial systems).
- NoSQL:
 - Most NoSQL databases follow the BASE (Basically Available, Soft state, Eventually consistent) model.

- Designed to provide high availability and partition tolerance, but may allow some degree of eventual consistency.

5. Query Language:

- SQL:
 - Uses structured query language (SQL) for querying the database (e.g., SELECT, INSERT, UPDATE, DELETE).
 - Well-defined and standardized.
- NoSQL:
 - Query languages vary based on the type of NoSQL database (e.g., MongoDB uses its own query language).
 - Queries can be less standardized across NoSQL systems.
 -

6. Use Cases:

- SQL:
 - Best suited for applications with complex querying needs, transactional systems, and when data consistency is paramount (e.g., banking, CRM systems).
- NoSQL:
 - Ideal for applications with large amounts of unstructured or semi-structured data, or when scalability and flexibility are more important than strict consistency (e.g., big data, real-time web apps, IoT).

7. Examples of Databases:

- SQL:
 - MySQL, PostgreSQL, SQLite, MS SQL Server, Oracle DB.
- NoSQL:
 - MongoDB, Cassandra, Couchbase, Redis, Neo4j.

Create User

```
CREATE USER username@hostname IDENTIFIED BY 'password';
CREATE USER username IDENTIFIED BY 'password'; -- The hostname is optional
```

Show all Users

```
SELECT user, host FROM mysql.user;
```

user	host
root	localhost
newuser	localhost
admin	%

Show Current User

```
SELECT USER();
Select current_user();
```

User Password Change

```
SET PASSWORD FOR 'mohits4'@'hostname' = PASSWORD('jtp12345'); -- older versions
-- new version
ALTER USER mohits4@hostname IDENTIFIED BY 'jtp123';

-- if current user logged-in:
SET PASSWORD = 'new_password';
-- After password change:
FLUSH PRIVILEGES;
```

Drop User

```
DROP USER mohits4@localhost;
--can also be used to remove more than one user accounts at once.
DROP USER john@localhost, peter@localhost;
```

Grant Privileges to the MySQL New User

1. **ALL PRIVILEGES**: It permits all privileges to a new user account.
2. **CREATE**: Allows the user to create new databases, tables, indexes, views, or stored procedures.
3. **DROP**: Allows the user to delete (drop) existing databases, tables, views, or other objects.
4. **DELETE**: It enables the user account to delete rows from a specific table.
5. **INSERT**: It enables the user account to insert rows into a specific table.
6. **SELECT**: It enables the user account to read a database.
7. **UPDATE**: It enables the user account to update table rows.

☐ Note:- Sometimes, you want to flush all the privileges of a user account for changes occurs immediately

```
FLUSH PRIVILEGES;
```

```
-- 1. The first asterisk (*) refers to all databases
-- 2. The second asterisk (*) refers to all tables
GRANT CREATE, SELECT ON database_name.* TO 'username'@'host';

-- If you want to give all privileges to a newly created user, execute the
following command.
GRANT ALL PRIVILEGES ON * . * TO username@hostname;

-- If you want to give specific privileges to a newly created user, execute the
following command.
GRANT CREATE, SELECT, INSERT ON * . * TO username@hostname;
```

Show Privileges

```
SHOW GRANTS for mohits4;
SHOW GRANTS FOR 'local_user'@'localhost';
-- if user logged in
SHOW GRANTS;
```

REVOKE Privileges

```
REVOKE ALL PRIVILEGES ON *.* FROM 'mohits4'@'hostname';
-- If you want to remove specific privileges to a newly created user
REVOKE SELECT ON *.* FROM 'mohits4'@'hostname';
-- Revoke **all privileges** on a specific **database**:
REVOKE ALL PRIVILEGES ON database_name.* FROM 'mohits4'@'hostname';
-- Revoke **SELECT privilege** on a **specific** **table**:
REVOKE SELECT ON database_name.table_name FROM 'mohits4'@'hostname';
```

What is a database?

- A database is an organized collection of data that allows efficient storage, retrieval, and management.

Show Database

```
SHOW DATABASES/SCHEMAS;
SHOW DATABASES [LIKE 'pattern' | WHERE expr];
SHOW DATABASES LIKE 'test%';
SELECT * FROM sys.databases;
EXEC sp_databases;

+-----+
| Database          |
+-----+
| employeesdb       |
| profile_fastapi   |
+-----+
```

Create Database

```
CREATE DATABASE databasename;
```

Rename Database

```
RENAME DATABASE old_database_name TO new_database_name
(OR)
ALTER DATABASE old_database MODIFY NAME = new_database
```

Drop Database

```
DROP DATABASE/SCHEMA database_name;
-- DROP DATABASE IF EXISTS Statement
DROP DATABASE IF EXISTS DatabaseName;
-- Deleting Multiple Databases
DROP DATABASE testDB3, testDB4;
```

Select Database

```
USE YourDatabaseName;
```

What is ACID property/SQL TRANSACTIONS?

- A transaction in SQL is a **sequence** of one or more SQL operations that are executed as a single unit.
- The goal of a transaction is to ensure that either all operations succeed or none of them do, maintaining the consistency of the database. Think of it like: "Do everything, or do nothing."
- (Transaction एक तरह का ब्लॉक है जिसमें कई SQL statements (जैसे INSERT, UPDATE, DELETE) एक साथ execute होते हैं। इसका मतलब है: या तो सारे काम पूरे होंगे, या कोई भी नहीं होगा।)

ACID Properties

- The ACID properties are four key principles that ensure database transactions are processed reliably and maintain data integrity.

1. A – Atomicity

- A transaction is treated as a single unit of work.
- Either all operations are completed, or none of them are.
- Example: During a bank transfer, if money is deducted from one account but cannot be added to the other, the entire transaction is rolled back.
 - Ya to transaction ka sara kaam hoga, ya kuch bhi nahi hoga.
 - All Operations Success OR All Operations Fail

2. C – Consistency

- A transaction brings the database from one valid state to another.
- It ensures that all database rules, constraints, and relationships remain valid.
- Example: An account balance should never violate defined constraints after a transaction.
 - Database transaction ke baad bhi saare rules follow hone chahiye.
 - Database invalid state me nahi jana chahiye.

3. I – Isolation

- Multiple transactions can occur at the same time without interfering with each other.
- Each transaction behaves as if it is running alone until it is completed.
- Example: Two users updating the same record simultaneously should not see inconsistent intermediate results.
 - Ek transaction dusre transaction ke beech me interfere nahi karega.
 - Database lock aur isolation levels use karta hai taaki dono transactions safely execute hon.

4. D – Durability

- Once a transaction is successfully committed, its changes are permanent.
- The data remains saved even if there is a power failure or system crash.
- Example: After confirming an online payment, the transaction remains recorded even if the server restarts.
 - Ek baar transaction commit ho gaya. To data permanently save ho gaya. Chahe System Crash, Server Restart, Power Failure Kuch bhi ho jaye. Data lose nahi hoga.

```
BEGIN TRANSACTION;
```

```
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
```

```
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
```

```
COMMIT;
```

```
-- If any of those updates fail (e.g., account not found), you can use:
```

```
ROLLBACK;
```

🎯 Difference between CHAR vs VARCHAR

- Both of these data types are used for characters.
- CHAR is a **fixed-length data type**, whereas VARCHAR is a **variable-length data type**.
- CHAR is suitable for **fixed-size values**, while VARCHAR is suitable for **values whose length can vary**.
- char occupies all the space and if space is remaining, then it fill all the blank space with "space". But in case of varchar, It takes only the required length & release remaining.

```
Char -> 10      | R | A | M | space | space | sapce | space | space | space |
space |
Varchar -> 10   | R | A | M |   |   |   |   |   |   |
| R | A | M |
```

- varchar is better than Char in term of space.
- char perform is better than varchar.
- Char max 256 characters, varchar 65535 characters.

```
-- CHAR Example
CREATE TABLE users (
    country_code CHAR(2)
);

INSERT INTO users (country_code)
VALUES ('IN'), ('US'), ('UK');

-- Yahan CHAR(2) suitable hai kyunki har country code ki length fixed 2 characters
hai.

--
-- varchar example
-- jaise name
CREATE TABLE users (
    name VARCHAR(50)
);
```

Difference between Delete, Truncate & Drop?

- DELETE removes selected rows and supports WHERE clause. TRUNCATE removes all rows from a table, keeps the structure, and is faster than DELETE. DROP removes the entire table including its structure, indexes, and data from the database. DELETE is DML, whereas TRUNCATE and DROP are DDL commands.

- **DELETE**

- Used to remove specific rows from a table.
- Supports WHERE clause.
- Row-by-row deletion happens.
- Triggers are fired.
- Can be rolled back (inside a transaction).
- Does not reset AUTO_INCREMENT.

```
DELETE FROM employees WHERE id = 10;
```

- **TRUNCATE**

- Removes all records from a table.
- Does not support WHERE.
- Faster than DELETE because it doesn't scan rows one by one.
- Resets AUTO_INCREMENT.
- Table structure remains intact.
- Usually does not fire DELETE triggers.

```
TRUNCATE TABLE employees;
```

- **DROP**

- Deletes the entire table permanently.
- Removes data, structure, indexes, constraints, and permissions.
- Table no longer exists after execution.
- Fastest operation.
- To use the table again, it must be recreated.

```
DROP TABLE employees;
```

Difference between WHERE and HAVING clauses?

- WHERE **filters individual records before grouping**, whereas HAVING **filters groups after GROUP BY**. HAVING is mainly **used to apply conditions on aggregate functions** such as COUNT, SUM, and AVG.

```
-- WHERE → row filtering
SELECT * FROM employees
WHERE salary > 50000;

-- HAVING → group filtering
SELECT department, COUNT(*) FROM employees
GROUP BY department
HAVING COUNT(*) > 2;
```

1. WHERE **filters rows**, while HAVING **filters groups**.
2. WHERE is applied **before** GROUP BY.
3. HAVING is applied **after** GROUP BY.
4. WHERE is generally used for **row-level conditions**.
5. HAVING is mainly used with aggregate functions like:
 1. COUNT()
 2. SUM()
 3. AVG()
 4. MAX()
 5. MIN()
6. WHERE cannot normally be used to filter an aggregate result directly.
7. HAVING can filter aggregate results.
8. WHERE can be used without GROUP BY.
9. HAVING is commonly used with GROUP BY.
10. Easy way to remember:
 1. WHERE → Row filter
 2. HAVING → Group filter

Example

1. WHERE

- Find employees whose salary is greater than 50,000:

id	name	department	salary
1	Amit	IT	50000
2	Rahul	IT	60000
3	Neha	HR	40000
4	Priya	HR	45000
5	Raj	IT	70000


```
SELECT * FROM employees WHERE salary > 50000;
```

1. HAVING

- Find departments where the average salary is greater than 50,000:

```
SELECT department, AVG(salary) AS avg_salary FROM employees  
GROUP BY department  
HAVING AVG(salary) > 50000;
```

Where VS Having

WHERE	HAVING
Filters rows	Filters groups
Used before GROUP BY	Used after GROUP BY
Cannot normally use aggregate conditions like COUNT(), AVG()	Used with aggregate functions
Example: WHERE salary > 50000	Example: HAVING AVG(salary) > 50000

Combining WHERE and HAVING:

```
SELECT department, COUNT(*) AS total FROM employees  
WHERE salary > 40000 -- Filters rows before GROUP BY  
GROUP BY department  
HAVING COUNT(*) > 2; -- Filters groups after GROUP BY
```

SQL Comments?

- There are typically two main types of SQL comments:

1. **Single-line Comments:** Started with two dashes (--)

```
-- This is a single-line comment  
SELECT * FROM employees; -- Retrieve all employee records
```

2. **Multi-line Comments:** Started with /* and ended with */

```
/* This is a  
   multi-line comment  
   explaining the query */  
SELECT id, name, email FROM employees;
```

What is Aggregate function?

```
-- Sum() :- The SUM() function returns the total sum of a numeric column.
SELECT SUM(column_name) FROM table_name;

-- AVG() :- The AVG() function returns the average value of a numeric column.
SELECT AVG(column_name) FROM table_name;

-- MAX() :- The MAX() function returns the largest value of the selected column.
SELECT MAX(column_name) FROM table_name;

-- Min() :- The MIN() function returns the smallest value of the selected column.
SELECT MIN(column_name) FROM table_name;

-- count() :- The COUNT() function returns the number of rows that matches a
specified criterion.
SELECT COUNT(column_name) FROM table_name;
```

What are constraints in MySQL?

- Constraints are **rules applied to table columns** to ensure the accuracy, consistency, and integrity of data in a database.
- Constraints are like rules on a form
 - "This field is required" → NOT NULL
 - "Must be unique" → UNIQUE
 - "Must be at least 18 years old" → CHECK

Common MySQL Constraints

Constraint	Purpose
NOT NULL	Column cannot contain NULL values
UNIQUE	All values in the column must be unique
PRIMARY KEY	Uniquely identifies each row (NOT NULL + UNIQUE)
FOREIGN KEY	Maintains relationship between tables
CHECK	Ensures values meet a condition
DEFAULT	Assigns a default value if none is provided
AUTO_INCREMENT	Automatically generates sequential numbers
CREATE INDEX	Used to create and retrieve data from the database very quickly

Wildcard Characters/Like Query?

- Wildcard characters are used with the **LIKE operator**. The LIKE operator is **used** in a **WHERE** clause to search for a specified pattern in a column.

Symbol	Description
%	Represents zero or more characters
_	Represents a single character

Some Example

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that starts with "a"
WHERE CustomerName LIKE '%a'	Finds any values that ends with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a_%_%'	Finds any values that starts with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that starts with "a" and ends with "o"

```
SELECT * FROM Customers WHERE City LIKE 'ber%';
```

[↑ TOP](#) [Back to Top](#)

Aliases

- AS is a keyword in SQL that allows you to rename a column or table using an alias.
- Aliases are used to give a table, or a column in a table, a temporary name.
- An alias is created with the **AS** keyword.

Alias Column Syntax:-

```
SELECT column_name AS alias_name FROM table_name;

-- Basic Column Alias
SELECT first_name AS name, last_name AS surname FROM employees;

-- Using quotes for aliases with spaces
SELECT
    first_name AS 'Employee First Name',
    last_name AS 'Employee Last Name',
    salary AS 'Monthly Compensation'
FROM
    employees;

-- Without AS keyword
SELECT first_name name, last_name surname FROM employees;

-- Simple Table Alias
SELECT e.first_name, e.last_name, d.department_name
FROM employees e JOIN departments d
ON e.department_id = d.department_id;

-- Calculation with Alias
SELECT
    product_name,
    price * quantity AS total_value,
    (price * quantity) * 1.1 AS total_with_tax
FROM products;

-- Concatenation Alias
SELECT
    CONCAT(first_name, ' ', last_name) AS full_name,
    email AS contact_email
FROM customers;

-- Subquery with Alias
SELECT
    (SELECT AVG(salary) FROM employees) AS avg_salary,
    (SELECT MAX(salary) FROM employees) AS max_salary;

-- Aggregate Function Aliases
```

```
SELECT
    department_id,
    AVG(salary) AS average_salary,
    COUNT(*) AS employee_count,
    MAX(salary) AS highest_salary
FROM employees
GROUP BY department_id;

-- Conditional Aliases
SELECT
    first_name,
    last_name,
    CASE
        WHEN salary < 50000 THEN 'Junior'
        WHEN salary BETWEEN 50000 AND 100000 THEN 'Mid-Level'
        ELSE 'Senior'
    END AS salary_category
FROM employees;
```

Primary Key?

- A PRIMARY KEY is a column or combination of columns that **uniquely identifies each record** in a database table.
- A Primary Key column **cannot have Null values**.
- A table can have only **one primary key** per table.
- When **multiple fields** are used as a primary key, they are called a **composite key**.

```
-- Create Primary Key
CREATE TABLE Students (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50),
    email VARCHAR(100)
);
-- (OR)
CREATE TABLE Students (
    student_id INT AUTO_INCREMENT,
    name VARCHAR(50),
    email VARCHAR(100),
    PRIMARY KEY (student_id)
);

-- Create Primary Key with multiple column
CREATE TABLE Order_Items (
    order_id INT,
    product_id INT,
    quantity INT,
    PRIMARY KEY (order_id, product_id) -- Multiple columns as primary key
);
```

Add primary Key

```
-- if primary key doesn't exists in the created table
ALTER TABLE table_name ADD PRIMARY KEY (Id)

-- For multiple column
ALTER table Employee ADD constraints PK_Employee PRIMARY KEY (column_name1,
column_name2);
-- (OR)
ALTER TABLE table_name ADD PRIMARY KEY (column1, column2);

-- Adding Primary Key with Auto-Increment
ALTER TABLE table_name MODIFY column_name INT AUTO_INCREMENT PRIMARY KEY;
```


Delete primary Key

```
ALTER TABLE table_name DROP PRIMARY KEY;

-- For multiple column
ALTER TABLE Employee DROP CONSTRAINT PK_Employee;
-- (OR)
ALTER TABLE table_name DROP PRIMARY KEY;
```

Ques. What Is Unique Key?

- A Unique Key is a constraint that ensures all values in a column or a combination of columns are **unique across all records** in a table.
- The Unique and Primary Key constraints both provide a guarantee for a column or set of columns.
- A Primary Key consist automatically has a unique constraint define on it.
 - Defining unique key

```
-- Single Column Unique Key
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    email VARCHAR(100) UNIQUE
);

-- Multiple Column Unique Key
CREATE TABLE Employees (
    id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    UNIQUE (first_name, last_name)
);
```

ALTER unique key

```
-- Single Column
ALTER table Employee ADD UNIQUE(column name);
-- (OR)
ALTER TABLE table_name ADD CONSTRAINT constraint_name UNIQUE (column_name);

-- Multiple Columns
ALTER TABLE table_name ADD CONSTRAINT constraint_name UNIQUE (column1, column2);
```

Drop unique key

```
ALTER TABLE Employee DROP CONSTRAINT Employee_ID;
-- (OR)
ALTER TABLE table_name DROP INDEX constraint_name;
```

Ques. What Is Foreign Key?

- A foreign key is a key used to link two tables together. This is something called a reference key.
- A column or set of columns in a table that references the PRIMARY KEY of another table.
- Foreign key is a column or a combination of columns whose values match a primary key in a different table.
- The relationship between two tables matches the primary key in one of the tables with a foreign key in the second table.

```
-- create Customers table
CREATE TABLE Customers (
  id INTEGER PRIMARY KEY,
  name VARCHAR(100),
  age INTEGER
);

-- create Products table
CREATE TABLE Products (
  customer_id INTEGER ,
  name VARCHAR(100),
  FOREIGN KEY (customer_id)
  REFERENCES Customers(id)
);

-- Here, the customer_id column in the Products table references the id column in
the Customers table.
```

- **One-to-One:-** Each record in one table connects to single record in another

```
CREATE TABLE Employee (
  emp_id INT PRIMARY KEY,
  passport_number INT,
  FOREIGN KEY (passport_number)
  REFERENCES Passport(passport_number)
);
```

- **One-to-Many:-** One record in parent table can relate to multiple records in child table

```
CREATE TABLE Orders (
  order_id INT PRIMARY KEY,
  customer_id INT,
  FOREIGN KEY (customer_id)
  REFERENCES Customers(customer_id)
);
```

- **Many-to-Many:-** Multiple records in both tables can relate to each other

```
CREATE TABLE StudentCourses (  
    student_id INT,  
    course_id INT,  
    PRIMARY KEY (student_id, course_id),  
    FOREIGN KEY (student_id) REFERENCES Students(student_id),  
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
);
```

Alter Foreign Key to Existing Table?

```
-- if primary key doesn't exists in the created table  
ALTER TABLE Employee ADD FOREIGN KEY (department_id) REFERENCES  
Department(department_id);  
(OR)  
ALTER TABLE `bookings` ADD CONSTRAINT `advance_bookings_user_id_foreign` FOREIGN  
KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE ON UPDATE CASCADE  
  
-- For multiple column  
ALTER TABLE Employee ADD CONSTRAINT FK_dept_id FOREIGN KEY (department_id)  
REFERENCES Department(department_id);
```

DROP a Foreign Key from the table

```
-- For single column/multiple column  
ALTER TABLE Employee DROP FOREIGN KEY FK_dept_id;
```

Ques. What is Composite Key?

- Composite key is **combination of two or more columns** that can **uniquely identify each row in the table**.
- composite key is also a primary key, but the difference is that it is made by the combination of more than one column to identify the particular row in the table.
- A composite key cannot be null.

```
CREATE TABLE student
(rollNumber INT,
name VARCHAR(30),
class VARCHAR(30),
section VARCHAR(1),
mobile VARCHAR(10),
PRIMARY KEY (rollNumber, mobile));
```

Types of Composite Keys

1. Composite Primary Key

```
CREATE TABLE StudentCourses (
    student_id INT,
    course_id INT,
    semester VARCHAR(10),
    PRIMARY KEY (student_id, course_id, semester)
);
```

2. Composite Unique Key

```
CREATE TABLE Employees (
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    department VARCHAR(50),
    UNIQUE (first_name, last_name, department)
);
```

Can any one field in a composite key be NULL?

- **Composite Primary Key:** No, because all columns that are part of a primary key are automatically NOT NULL.
- **Composite Unique Key:** Yes, in MySQL NULL values can be allowed depending on the constraint and database behaviour.

Ques. Difference between Primary Key & Unique Key?

Primary Key	Unique Key
A table can have only one primary key	A table can have more than one unique key
It does not allow null values	Allows null values
Primary key Can be made foreign key into another table	In SQL server, unique key Can be made foreign key into another table
By default it adds a clustered index	By default it adds a unique non-clustered index
Primary key support auto increment value.	Unique constraint does not support auto increment value.

Ques. Difference between Primary Key & Foreign Key?

Primary Key	Foreign Key
A table can have only one primary key	A table can have more than one foreign key
Primary key uniquely identified a record in the table	Foreign key is a field in the table that is primary key in another table
It does not allow null values	Allows null values
Duplicate not allowed	Duplicate allowed
Primary key support auto increment value	Foreign key do not automatically create an index.

Ques. What Is Joins?

- A JOIN is a method used to **combine rows from two or more tables** based on a related column between them.
- MySQL JOINS are used with SELECT statement.

Many types of MySQL joins:

1. Self Join
2. Inner Join
3. Left JOIN
4. Right JOIN
5. Full Join
6. Outer Join
7. Cross Join

Self Join

- A self join **connects a table to itself**. Used when you want to compare rows within the same table.

Customers table:

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

-- Example

```
SELECT c1.NAME AS Customer1, c2.NAME AS Customer2, c1.SALARY FROM Customers c1
JOIN Customers c2 ON c1.SALARY = c2.SALARY AND c1.ID < c2.ID;
```

-- Output:

Customer1	Customer2	SALARY
Ramesh	kaushik	2000.00

INNER JOIN

- The MySQL Inner Join is used to returns only those results from the tables that **match** the specified condition and hides other rows and columns.
- Inner join: Inner join return rows when there is at least one match of rows between the tables.

Customers table:

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Order table:

OID	DATE	CUSTOMER_ID	AMOUNT
101	2009-11-20 00:00:00	2	1560
103	2008-05-20 00:00:00	4	2060

```
SELECT customers.name, customers.age, customers.salary, order.date
FROM customers INNER JOIN order
ON customers.id = order.CUSTOMER_ID;
```

NAME	AGE	SALARY	DATE
Khilan	25	1500.00	2009-11-20 00:00:00
Chaitali	25	6500.00	2008-05-20 00:00:00

-- Example 2

Order table:

OID	DATE	CUSTOMER_ID	AMOUNT
102	2009-10-08 00:00:00	3	3000
100	2009-10-08 00:00:00	3	1500
101	2009-11-20 00:00:00	2	1560
103	2008-05-20 00:00:00	4	2060

-- Output:-

```
SQL> SELECT ID, NAME, AMOUNT, DATE FROM CUSTOMERS
INNER JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID;
```



```
+-----+-----+-----+-----+
| ID | NAME      | AGE | AMOUNT |
+-----+-----+-----+-----+
| 3 | kaushik   | 23  | 3000   |
| 3 | kaushik   | 23  | 1500   |
| 2 | Khilan    | 25  | 1560   |
| 4 | Chaitali  | 25  | 2060   |
+-----+-----+-----+-----+
```

Left JOIN/LEFT OUTER JOIN

- The LEFT JOIN keyword returns all records from the left table (the table listed first), and the matching records (if any) from the right table (table2).

CUSTOMERS Table

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Orders Table

OID	DATE	CUSTOMER_ID	AMOUNT
102	2009-10-08 00:00:00	3	3000
100	2009-10-08 00:00:00	3	1500
101	2009-11-20 00:00:00	2	1560
103	2008-05-20 00:00:00	4	2060

```
SQL> SELECT ID, NAME, AMOUNT, DATE FROM CUSTOMERS LEFT JOIN ORDERS
      ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID;
```

Result:-

ID	NAME	AMOUNT	DATE
1	Ramesh	NULL	NULL
2	Khilan	1560	2009-11-20 00:00:00
3	kaushik	3000	2009-10-08 00:00:00
3	kaushik	1500	2009-10-08 00:00:00
4	Chaitali	2060	2008-05-20 00:00:00
5	Hardik	NULL	NULL
6	Komal	NULL	NULL
7	Muffy	NULL	NULL

Right JOIN

- The RIGHT JOIN keyword returns **all records** from the **right table** (table2), and the **matching records** (if any) from the **left table** (table1).

-- Customers Table

ID	NAME
1	Ramesh
2	Khilan
3	Kaushik

-- Orders Table

OrderID	CustomerID
101	2
102	3
103	4

-- RIGHT JOIN Result

```
SELECT Customers.NAME, Orders.OrderID FROM Customers RIGHT JOIN Orders  
ON Customers.ID = Orders.CustomerID;
```

NAME	OrderID
Khilan	101
Kaushik	102
NULL	103

Outer join

- Returns all rows from both tables, Matching and non-matching rows.
- NULL values where no match exists

```
SELECT column_list
FROM table1
FULL OUTER JOIN table2 ON table1.column = table2.column;
```

-- Example query
employees:

employee_id	employee_name	department_id
1	John Smith	101
2	Mary Johnson	102
3	Sam Brown	103

departments:

department_id	department_name
101	HR
102	Finance
104	Marketing

```
SELECT employees.employee_id, employees.employee_name, departments.department_name
FROM employees
FULL OUTER JOIN departments ON employees.department_id =
departments.department_id;
```

employee_id	employee_name	department_name
1	John Smith	HR
2	Mary Johnson	Finance
3	Sam Brown	NULL
NULL	NULL	Marketing

CROSS Join

- The CROSS JOIN keyword returns all records from both tables (table1 and table2).
- If you have a Products table with 2 products and a Colors table with 2 colors, a CROSS JOIN would return a result set with 6 combinations (3 * 2):

-- Products table

ProductID	ProductName
1	T-Shirt
2	Jeans
2	Cap

-- Colors table

ColorID	Color
1	Red
2	Blue

-- Output:-

```
SELECT Products.ProductName, Colors.Color
FROM Products
CROSS JOIN Colors;
```

ProductName	Color
T-Shirt	Red
T-Shirt	Blue
Jeans	Red
Jeans	Blue
Cap	Red
Cap	Blue

Full Join/FULL OUTER JOIN

- A FULL JOIN (also called a FULL OUTER JOIN) returns all rows when there is a match in either left (table1) or right (table2) table. It returns all records from both tables, and the result set will have NULL values for columns where there is no match.

Key Points:

- Rows from both tables are included even if there is no match.
- If there is no match, the columns from the table that doesn't have a match will contain NULL.
- This join is useful when you want to retrieve all records, whether or not there's a match in both tables.

-- Employees Table:

EmployeeID	EmployeeName	DepartmentID
1	John	10
2	Jane	20
3	Mike	30
4	Sara	NULL

-- Departments Table:

DepartmentID	DepartmentName
10	HR
20	IT
30	Sales
40	Marketing

-- Output:-

```
SELECT Employees.EmployeeID, Employees.EmployeeName, Employees.DepartmentID,
Departments.DepartmentName
```

```
FROM Employees
```

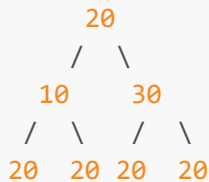
```
FULL JOIN Departments
```

```
ON Employees.DepartmentID = Departments.DepartmentID;
```

EmployeeID	EmployeeName	DepartmentID	DepartmentName
1	John	10	HR
2	Jane	20	IT
3	Mike	30	Sales
4	Sara	NULL	NULL
NULL	NULL	40	Marketing

What is Index?

- An index is used to enhance the performance of SQL Queries. It allows the database to find data quickly and efficiently by using Row ID, avoiding full table scans.
- Indexes can be created on one or more columns of a table.
- Index allows the database application to find data fast, without reading the whole table.
- An index can be created in a table to find data more quickly and efficiently.
- **How It Works:-** Behind the scenes, an index is usually implemented as a B-tree or similar structure.



Types of Indexes in MySQL

1. Primary Key Index

1. Automatically created when you define a PRIMARY KEY on a table.
2. Ensures that the key values are unique and that no null values are allowed in the indexed columns.
3. This is the most important index, as it determines how rows are physically stored in the table (in the clustered index format).

2. Unique Index

1. Similar to the primary key, but it allows for one or more NULL values in the indexed column(s).
2. It enforces uniqueness in the indexed columns, meaning no two rows can have the same value in that column (except NULL values).

3. Regular (Non-Unique) Index

1. The most common type of index used to speed up query performance.
2. Does not enforce uniqueness, but allows MySQL to search the indexed columns faster.
3. Often created on columns frequently used in WHERE, ORDER BY, GROUP BY, or JOIN conditions.

4. Full-Text Index

1. Used for full-text searches in text-based columns.
2. Works with MATCH() and AGAINST() to find words or phrases within large text fields. Commonly used for columns of type TEXT or VARCHAR.
3. MySQL supports full-text indexes in MyISAM and InnoDB (with some limitations in InnoDB).

5. Spatial Index

1. Used for indexing spatial data types (e.g., POINT, LINESTRING, POLYGON).
2. Typically used in geographical data queries, such as for finding points within a certain radius or performing complex spatial operations.

6. Composite Index

1. An index on multiple columns.
2. Useful when queries often use multiple columns in the WHERE clause, JOIN, or ORDER BY.
3. The order of the columns in the index matters; the index is most efficient if the query uses the columns in the same order as the index.

Types of Indexes?

1. **Single-column index**:- on one column

```
CREATE INDEX idx_column_name ON Sales (column_name);
```

2. **Composite/Multi-column indexes**: on multiple columns

```
CREATE INDEX idx_multiple_columns ON employees(column_name1, column_name2);
```

3. **Unique Index**:-

```
CREATE UNIQUE INDEX idx_email_unique ON employees(email);
```

4. **Full-Text Index** :- Designed for efficient text search in large text fields (e.g., for searching keywords in articles, product descriptions, etc.)

```
CREATE FULLTEXT INDEX idx_description ON products(description);
```

Show Index

```
show index from table_name
```

Unique Indexes

- A Unique Index is a database index that ensures the uniqueness of values in one or more columns of a database table.
- This index ensures that no two rows in the Employees table have the same employee_id(Column_name), which maintains data integrity and prevents duplicate entries.
- The SQL Unique Index ensures that no two rows in the indexed columns of a table have the same values (no duplicate values allowed).

* Behind the scenes, when a new record is inserted, the database would:

* **Check** the Index: It looks up the email in the index (which could be a B-tree or hash table) to find if that email already exists.

* The database will **search** the index, and it will quickly identify whether the email you're trying to insert is already in the table.

* **Check for Duplicates:**

* For example, if we try to insert a new row:

* **INSERT INTO** users (Name, Email) VALUES ('David', 'alice@example.com');

* The database will search the unique index for the email `alice@example.com`. Since this email already exists in the index, the insert will fail with an error like:

```
"Duplicate entry 'alice@example.com' for key 'idx_unique_email'".
```

```
-- Single column unique index
CREATE UNIQUE INDEX idx_email ON users(email);

-- Composite unique index
CREATE UNIQUE INDEX idx_name_age ON employees(last_name, first_name);
```

Alter/Modify an Index

```
-- Rename Index name
ALTER TABLE table_name
RENAME INDEX old_index_name TO new_index_name;

-- Modify Index name
ALTER TABLE table_name DROP INDEX existing_index,
ADD INDEX new_index (column1, column2);

-- Rebuild Index
ALTER INDEX index_name
ON table_name REBUILD;

-- Reorganize Index
ALTER INDEX index_name
ON table_name REORGANIZE;

-- Disable Index
ALTER INDEX index_name
ON table_name DISABLE;

-- Change Index Properties
ALTER INDEX index_name
ON table_name
SET (
    STATISTICS_NORECOMPUTE = OFF,
    ALLOW_ROW_LOCKS = ON,
    ALLOW_PAGE_LOCKS = ON
);
```

Drop Index

```
Drop index index_name on table_name
(OR) DROP INDEX table_name.index_name;
```

```
-- DROP INDEX with IF EXISTS
DROP INDEX IF EXISTS index_name
ON table_name;

ALTER TABLE table_name DROP CONSTRAINT constraint_name;
```

Cluster Index

- A clustered index is an index where the data in the table is physically stored in the order of the indexed column. In other words, the rows of the table are sorted on disk based on the values in the indexed column. A table can have only one clustered index because the data can only be sorted in one way.
- **Example:** If you create a clustered index on the EmployeeID column, the table rows will be stored on disk in ascending order of EmployeeID. The clustered index itself dictates the physical arrangement of the data.
- jab kisi table par hum primary key lagate hai to wo cluster index ban jata hai.

Non cluster index

- A non-clustered index is an index where the data in the table is stored independently of the index. It creates a separate structure that contains the indexed column(s) and pointers to the actual rows in the table. A table can have multiple non-clustered indexes, allowing for efficient lookups on different columns without changing the physical order of the data.
- **Example:** If you create a non-clustered index on the Department column, the index will store the department values along with pointers to the corresponding rows in the table, but the table data itself remains in its original order.
- table mai jis column par select command sabse jyda chalte hai to us column par hum non cluster index bna lete hai.

Ques. What is the difference between cluster and non cluster index?

Cluster index	Non cluster index
Data rows are stored in the order of the index.	Data rows are not sorted in any particular order.
Only one clustered index per table.	Multiple non-clustered indexes can exist per table.
Created by default for the primary key.	Must be explicitly created (e.g., for specific queries).

[Back to Top](#)

What is View?

- A view is a **virtual table** based on the result set of a SELECT query.
- It does not store data itself but provides a way to look at data from one or more tables in a structured and reusable manner.
- It **behaves** like a **table** but **doesn't store** the **data physically**.

Why use Views?

- Security provide karti hai (sab columns hide kar sakte hain).
- Complex queries ko simple banati hai.
- Reusable query hoti hai.
- Data abstraction provide karti hai.
- Multiple tables ko ek virtual table ki tarah dikhaya ja sakta hai.

Create view

```
Create view view_name As
Select column1, column2
From table_name
Where [condition];
```

Show view

```
SELECT * FROM view_name;
```

Alter view

- CREATE OR REPLACE VIEW Syntax

```
CREATE OR REPLACE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Deleted view

- A view is **deleted** with the **DROP VIEW statement**.

```
DROP VIEW view_name
```

```
DROP VIEW IF EXISTS view_name;
```

Can we insert data into a View?

- Haan, lekin sirf simple views me.

Can we create a View on another View?

- haa

```
CREATE VIEW view2 AS  
SELECT * FROM view1;
```

Why are Views used in real projects?

- Users ko sirf required columns dikhaye ja sakte hain bina original table ka access diye.
- Security, code reusability, complex query simplification, aur data abstraction ke liye Views use ki jaati hain.

What Is Union & Union All?

- Both UNION and UNION ALL Operator combine rows from result sets into a single result set.

UNION

- The union operator **combines** the results of two or more select statements by **removing duplicate rows**.
- The columns and the data types must be the same in select statements.

ID	NAME
1	Ramesh
2	Khilan
3	kaushik

ID	NAME
3	kaushik
4	Mohit
5	abhay

```
Select Column1, Column2, Column3 from Table A
UNION
Select Column1, Column2, Column3 from Table B
```

ID	NAME
1	Ramesh
2	Khilan
3	kaushik
4	Mohit
5	abhay

UNION ALL

- The UNION operator selects only distinct values by default. To **allow duplicate values**, use UNION ALL

```
Select Column1, Column2, Column3 from Table A
UNION ALL
Select Column1, Column2, Column3 from Table B
```

```
+----+-----+
| ID | NAME      |
+----+-----+
|  1 | Ramesh    |
|  2 | Khilan    |
|  3 | kaushik   |
|  3 | kaushik   |
|  4 | Mohit     |
|  5 | abhay     |
+----+-----+
```

Difference between Union & Union All?

Union	Union All
Union removes duplicate rows.	Union All does not remove the duplicate rows.
Union uses a distinct sort	Union All does not use a distinct sort
Union can't work with a column that has a text data type.	Union All can work with all data type column.

What is MINUS?

- MINUS operator will return only those rows which are **unique(distinct)** in only first SELECT query and not those rows which are **common to both first and second** SELECT queries.

```
-- Employees
| EmpID | EmpName | Department |
| -----|-----|-----|
| 1      | Alice   | HR         |
| 2      | Bob     | IT         |
| 3      | Charlie | Finance    |
| 4      | David   | IT         |
| 5      | Eve     | Marketing  |

-- Managers
| EmpID | EmpName | Department |
| -----|-----|-----|
| 2      | Bob     | IT         |
| 4      | David   | IT         |
| 6      | Frank   | Sales      |
| 7      | Grace   | Marketing  |

-- output:-
SELECT EmpID, EmpName, Department FROM Employees
MINUS
SELECT EmpID, EmpName, Department FROM Managers;
| EmpID | EmpName | Department |
| -----|-----|-----|
| 1      | Alice   | HR         |
| 3      | Charlie | Finance    |
| 5      | Eve     | Marketing  |
```

Key characteristics of the MINUS operator:

- Returns distinct rows:** Only unique rows from the first query that are not found in the second are returned.
- Requires compatible SELECT statements:** Both SELECT statements involved in the MINUS operation must have the same number of columns, and the corresponding columns must have compatible data types and be in the same order.
- Note:-** While MINUS is a standard SQL operator supported by many database systems (like Oracle, PostgreSQL), **MySQL does not** directly **support** the MINUS operator.

Achieving the MINUS functionality in MySQL:

- We can achieve the same results as MINUS in MySQL using various techniques, most commonly by combining **LEFT JOIN** and **WHERE** clauses, or using **subqueries** with **NOT IN** or **NOT EXISTS**.

1. Using LEFT JOIN

```
SELECT a.*
FROM table_a AS a
LEFT JOIN table_b AS b ON a.id = b.id
WHERE b.id IS NULL;
```

2. Using NOT IN

```
SELECT *
FROM table_a
WHERE id NOT IN (SELECT id FROM table_b);
```

3. Using NOT EXISTS

```
SELECT *
FROM table_a AS a
WHERE NOT EXISTS (SELECT 1 FROM table_b AS b WHERE a.id = b.id);
```

table_a

```
id  value
1   Apple
2   Banana
3   Cherry
4   Date
```

table_b

```
id  value
2   Banana
4   Date
```

-----Output:-----

```
id  value
1   Apple
3   Cherry
```

What is EXCEPT?

- same as minus nothing different.

What is Intersect?

- The INTERSECT statement will return only those rows that are **identical/common** to both of the SELECT statements from two or more tables

```
-- Employees                                -- Managers
| EmpID | EmpName | Department | | EmpID | EmpName | Department | | | | |
| -----| -|-----| -|-----| | -----| -|-----| -|-----|
| 1      | Alice   | HR         | | 2      | Bob     | IT         |
| 2      | Bob     | IT         | | 4      | David   | IT         |
```


3	Charlie	Finance		6	Frank	Sales	
4	David	IT		7	Grace	Marketing	
5	Eve	Marketing					

-- Output

SELECT EmpID, EmpName, Department FROM Employees

INTERSECT

SELECT EmpID, EmpName, Department FROM Managers;

EmpID	EmpName	Department	
-----	-----	-----	
2	Bob	IT	
4	David	IT	